modern web application. And I'll talk about the difference between "in theory" and "in practice."

If you use this process, you'll find that it changes the design of your code.

Software design is a tricky thing to pin down. We use the term all the time without really defining it. For our purposes in this book, *design* is anything in the code structure that goes beyond the logical correctness of the code. You can design software for many different reasons—optimization for speed, clarity of naming, robustness against errors, resistance to change, ease of maintenance, and so on.

Test-driven development enables you to design your software continuously and in small steps, allowing the design to emerge as the code changes.

Specifically, design happens at three different points in the test-driven process:

- When you decide which test to write next, you're making a claim about what functionality your code should have. This frequently involves thinking about how to add that functionality to the existing code, which is a design question.
- As you write a test, you're designing the interaction between the test and the code, which is also the interaction between the part of the code under test and the rest of the application. This part of the process is used to create the API you want the code to have.
- After the test passes, you refactor, identifying duplication, missing abstractions, and other ways the code's design can be improved.

Prescription 1	Use the TDD process to create and adjust your code's design in small, incremental steps.

Continually aligning your code to the tests tends to result in code that is made up of small methods, each of which does one thing. These methods tend to be loosely coupled and have minimal side effects. As it happens, the hallmarks of easy-to-change code include small methods that do one thing, are loosely coupled, and have minimal side effects.

I used to think it was a coincidence that tested code and easy-to-change code have similar structures, but I've realized the commonality is a direct side effect of building the code in tandem with the tests. In essence, the tests act as a universal client for the entire codebase, guiding all the code to have clean interactions between parts because the tests, acting as a third-party interloper, have to get in between all the parts of the code to work. Metaphorically,